MEDIA PEMBELAJARAN INFORMATIKA FASE E (KELAS X SMK)

Konsep Dasar Struktur Data & Algoritma Standar Pemrograman

Bahan Ajar Lengkap Pertemuan 11 s.d. 13 — Array, Stack, Queue, Bubble Sort, Binary Search, & Dictionary

Penyusun: Ikhsan Qadri, S.Pd.

Mata Pelajaran: Informatika | Elemen: Berpikir Komputasional | Modul Pembelajaran Mandiri

Cara Data Disimpan Menentukan Cara Data Diakses dalam Pemrograman

Pemahaman Kunci: Struktur data adalah cara mengatur dan menyimpan data di dalam program agar dapat diakses dan diolah secara efisien. Cara menyimpan menentukan data mana yang keluar lebih dulu, kecepatan akses, dan kemudahan perubahan.

Rak Berlaci Bernomor

Struktur: ARRAY

Aturan: Ambil Lewat Nomor (Indeks)

Kamu bisa langsung membuka laci nomor 5 tanpa perlu membuka laci nomor 1 sampai 4 terlebih dahulu.

Tumpukan Piring Kantin

Struktur: STACK (LIFO)

Aturan: Terakhir Masuk, Pertama Keluar

Piring yang kamu ambil selalu piring yang paling terakhir diletakkan di atas tumpukan.

Antrean di Kasir

Struktur: QUEUE (FIFO)

Aturan: Pertama Masuk, Pertama Keluar

Orang yang dilayani dan keluar dari antrean terlebih dahulu adalah orang yang datang paling awal.

Array Memungkinkan Akses Data Cepat Melalui Posisi Indeks

Konsep Utama Array

• Array adalah kumpulan data berurutan yang memiliki nomor posisi yang disebut indeks.

• Dalam Python (list), indeks dimulai dari 0 (bukan 1). Jika ada n elemen, indeks terakhir adalah n - 1.

• Memanggil data ke-k: gunakan indeks k - 1 (misal data ke-3 berada pada indeks 2).

• Operasi Dasar: Baca (nilai[2]), Ubah (nilai[1]=78), Tambah (nilai.append(85)), Hapus (nilai.pop(0)), Panjang (len(nilai)).

• Hati-hati: Akses indeks di luar rentang (misal indeks 10 pada array 3 elemen) memicu IndexError!

Contoh Kode Python & Karakteristik

```
nilai = [80, 75, 90]
print(nilai[0])    # 80 (elemen ke-1)
print(nilai[2])    # 90 (elemen ke-3)
nilai[1] = 78      # mengubah isi indeks 1
nilai.append(85)   # tambah 85 di akhir
print(len(nilai)) # 4 (banyak elemen)
```

✓ Kelebihan: Akses lewat indeks sangat cepat (langsung ke posisi memory), sederhana & cocok untuk data berurutan bertipe sama.

✕ Keterbatasan: Menyisipkan/menghapus di tengah menuntut elemen lain bergeser. Posisi indeks tidak memuat arti nama label.

Stack Mengakses Data dari Ujung Puncak dengan Prinsip LIFO

Aturan & Operasi Stack

- push(x): Menaruh data x di puncak (top) stack.
- pop(): Mengambil dan menghapus data di puncak stack.
- peek() / top: Melihat data di puncak tanpa menghapusnya.
- isEmpty(): Memeriksa apakah stack dalam keadaan kosong.

⚠️ **Awas: Melakukan pop() pada stack kosong menyebabkan kesalahan underflow!**

```
Stack di Python:
stack = []
stack.append('A')  # push(A)
stack.append('B')  # push(B)
print(stack.pop()) # pop() -> 'B'
```

Penelusuran Operasi (Tracing LKPD 11)

Urutan Operasi: push(A), push(B), push(C), pop(), pop()

1. push(A) -> Stack: [A] | Keluar: -
2. push(B) -> Stack: [A, B] | Keluar: -
3. push(C) -> Stack: [A, B, C] | Keluar: -
4. pop() -> Stack: [A, B] | Keluar: C (Puncak)
5. pop() -> Stack: [A] | Keluar: B (Puncak)

- Data tersisa: A (pada indeks 0 / dasar tumpukan).
- Urutan keluar: C lalu B (kebalikan dari urutan masuk!).

Prinsip LIFO Stack Sangat Cocok untuk Membatalkan Aksi (Undo)

Mengapa Stack Cocok untuk Fitur Undo?

Saat mengedit dokumen, setiap perubahan disimpan ke dalam stack riwayat. Perubahan TERAKHIR yang dilakukan pengguna adalah hal PERTAMA yang dibatalkan saat menekan tombol Undo. Ini persis perilaku LIFO.

```
Simulasi: Ketik 'Halo' -> Tebalkan teks -> Ganti warna | Stack: [Ketik, Tebalkan, Warna]
-> Undo 1: 'Ganti warna' dibatalkan | Stack tersisa: [Ketik, Tebalkan]
-> Undo 2: 'Tebalkan' dibatalkan | Stack tersisa: [Ketik 'Halo']
```

Tombol Back Browser

Menyimpan riwayat halaman web. Halaman yang terakhir dibuka adalah yang pertama dikembalikan saat menekan Back.

Pemanggilan Fungsi (Call Stack)

Program menyimpan posisi pemanggilan fungsi. Fungsi terakhir yang dipanggil harus selesai dieksekusi lebih dulu.

Pemeriksaan Tanda Kurung

Memastikan pasangannya cocok pada ekspresi matematika/kodingan, misalnya pada ({[]}).

Queue Menjamin Keadilan Layanan dengan Prinsip First In, First Out

Aturan & Operasi Queue

• enqueue(x): Menambah data x di belakang (rear) antrean.

• dequeue(): Mengambil dan menghapus data di depan (front) antrean.

• front: Melihat data terdepan tanpa menghapusnya.

• isEmpty(): Memeriksa apakah antrean kosong.

```
Queue di Python (menggunakan deque):
from collections import deque
antrean = deque()
antrean.append('A')      # enqueue(A)
antrean.append('B')      # enqueue(B)
print(antrean.popleft())# dequeue() -> 'A'
```

Tracing & Aplikasi Utama Queue

Penelusuran Operasi Queue:
enqueue(A), enqueue(B), enqueue(C), dequeue(), dequeue()

1. enqueue(A) -> Queue: [A] | Keluar: -
2. enqueue(B) -> Queue: [A, B] | Keluar: -
3. enqueue(C) -> Queue: [A, B, C] | Keluar: -
4. dequeue() -> Queue: [B, C] | Keluar: A (Depan)
5. dequeue() -> Queue: [C] | Keluar: B (Depan)

• Data tersisa: C (di posisi belakang).
• Urutan keluar: A lalu B (sama dengan urutan masuk!).

Aplikasi Utama: Antrean Cetak (Print Spooler), Antrean Kasir/Sistem Pelanggan, & Antrean Tugas Sistem.

Perbedaan Arah Pengambilan Menentukan Urutan Keluaran Data

Aspek Perbandingan	Stack (Tumpukan)	Queue (Antrean)
Prinsip Utama	LIFO (Last In, First Out)	FIFO (First In, First Out)
Tempat Penambahan Data	Puncak (Top)	Belakang (Rear)
Tempat Pengambilan Data	Puncak (Top)	Depan (Front)
Istilah Operasi	push() dan pop()	enqueue() dan dequeue()
Skenario Penggunaan Cocok	Fitur Undo, Tombol Back, Call Stack	Antrean Cetak (Print Spooler), Kasir

Eksperimen: Masukkan A, B, C secara berurutan, lalu keluarkan dua kali.

- Hasil pada STACK : Data keluar = C, B | Data tersisa = A
- Hasil pada QUEUE : Data keluar = A, B | Data tersisa = C

Kesimpulan: Urutan operasi yang persis sama menghasilkan keluaran dan sisa data yang berbeda total!

Linked List Mengorbankan Kecepatan Akses Demi Kemudahan Diselipi Data

Konsep Linked List (Rantai Node)

Linked list adalah struktur data yang tiap elemennya (node) menyimpan data dan penunjuk (pointer) ke node berikutnya. Node tidak harus berdekatan di memori komputer, berbeda dengan array yang rapat.

Aspek Evaluasi	Array	Linked List
Akses Elemen ke-k	Sangat Cepat (langsung via indeks)	Lambat (harus menelusuri dari awal)
Sisip / Hapus di Tengah	Lambat (perlu menggeser banyak elemen)	Cukup mengubah penunjuk (pointer)
Penggunaan Memori	Hemat (data tersusun rapat)	Lebih boros (ada tambahan memori pointer)
Ukuran Struktur	Cenderung tetap / fleksibel terbatas	Sangat mudah bertambah dan berkurang
Skenario Penggunaan Cocok	Daftar nama siswa yang sering dibaca urut	Daftar putar lagu yang sering disisipi di tengah

Efisiensi Algoritma Diukur dari Jumlah Langkah Perbandingan & Pertukaran

Apa itu Algoritma?

Algoritma adalah urutan langkah yang jelas dan berhingga untuk menyelesaikan suatu masalah.

4 Ciri Algoritma yang Baik:

✓ Jelas: Setiap langkah tidak ambigu.

✓ Berhingga: Memiliki titik henti yang pasti.

✓ Memiliki Masukan & Keluaran: Mengolah data awal menjadi hasil.

✓ Efektif: Langkah-langkahnya dapat dilakukan oleh komputer.

Mengapa Menghitung Langkah?

• Untuk membandingkan dua algoritma, kita tidak mengukur satuan detik/waktu, karena waktu eksekusi sangat bergantung pada kecepatan spesifikasi komputer.

• Ukuran efisiensi sejati dihitung dari jumlah LANGKAH (misalnya jumlah perbandingan dan jumlah pertukaran posisi data).

• Semakin sedikit langkah untuk menyelesaikan kumpulan data yang sama, semakin efisien algoritma tersebut.

• Aplikasi Nyata: Papan peringkat nilai, pencarian nama kontak, katalog toko online, hingga mesin pencari Google!

Bubble Sort Mengurutkan Data dengan Menukar Pasangan Bersebelahan

Ide Dasar & Langkah-Langkah

Disebut 'gelembung' karena elemen terbesar perlahan 'mengapung' ke posisi ujung kanan pada tiap putaran.

1. Mulai dari elemen paling kiri.
2. Bandingkan elemen dengan tetangga di sebelah kanannya.
3. Jika elemen kiri > kanan, TUKAR keduanya (ascending).
4. Lanjutkan ke pasangan berikutnya sampai ujung kanan.
5. Ulangi putaran dari awal. Jika dalam 1 putaran tidak ada pertukaran, data SUDAH TERURUT dan proses berhenti!

Simulasi [7, 3, 5, 1] & Analisis

Awal: [7, 3, 5, 1]
• Putaran 1: (7,3) tukar -> [3,7,5,1]; (7,5) tukar -> [3,5,7,1]; (7,1) tukar -> [3,5,1,7] (3 Pertukaran)
• Putaran 2: (3,5) tidak; (5,1) tukar -> [3,1,5,7]; (5,7) tidak (1 Pertukaran)
• Putaran 3: (3,1) tukar -> [1,3,5,7]; (3,5) tidak; (5,7) tidak (1 Pertukaran)
• Putaran 4: Tidak ada pertukaran -> Selesai! Total Pertukaran = 5.

Analisis Kasus Terburuk: n(n - 1) ÷ 2 perbandingan.
• 4 data = 6 langkah, 16 data = 120 langkah!
• Bubble Sort mudah dipahami tetapi LAMBAT untuk data besar.

Pencarian Linear Memeriksa Elemen Satu per Satu Tanpa Syarat Urutan

Cara Kerja & Contoh Tracing

Pencarian linear memeriksa elemen satu per satu dari awal (indeks 0) sampai target ditemukan atau seluruh data habis diperiksa.

```
Contoh: Mencari angka 23 pada [15, 40, 8, 23, 4]
• Langkah 1: Cek 15 (Bukan)
• Langkah 2: Cek 40 (Bukan)
• Langkah 3: Cek 8  (Bukan)
• Langkah 4: Cek 23 (DITEMUKAN di indeks 3!)
```

Analisis Kasus & Keunggulan

• Kasus Terbaik (Best Case): 1 Langkah (Target berada pada elemen paling awal).

• Kasus Terburuk (Worst Case): n Langkah (Target berada pada elemen paling akhir atau tidak ada dalam data).

• Keunggulan Utama: Sangat sederhana dan dapat bekerja pada data acak/tidak terurut.

• Kelemahan Utama: Sangat lambat pada data bervolume besar (harus memeriksa ribuan data satu per satu).

Binary Search Membelah Dua Ruang Cari dengan Syarat Data Wajib Terurut

Ide Dasar Membelah Dua

SYARAT MUTLAK: Data HARUS TERURUT terlebih dahulu!

1. Tentukan batas bawah (low) dan batas atas (high).
2. Hitung posisi tengah: mid = (low + high) // 2.
3. Bandingkan target dengan nilai di mid:
   - Sama -> Ditemukan!
   - Target < mid -> Buang bagian kanan (high = mid - 1).
   - Target > mid -> Buang bagian kiri (low = mid + 1).
4. Ulangi sampai ditemukan atau low > high.

⚠️ Mengapa Gagal pada Data Acak?

Binary Search membuang separuh data berdasarkan satu perbandingan di titik tengah.

Keputusan membuang separuh ruang cari HANYA SAH jika semua nilai di kiri pasti lebih kecil dan di kanan pasti lebih besar.

Jika data acak, bagian yang dibuang bisa saja memuat target! Akibatnya Binary Search memberikan jawaban SALAH (dianggap tidak ada padahal ada), bukan sekadar lambat!

Binary Search Sangat Unggul Pada Data Besar Terurut (1 Juta Data = 20 Langkah)

Aspek Perbandingan	Pencarian Linear	Binary Search (Pencarian Biner)
Syarat Kondisi Data	Tidak perlu terurut (data acak)	HARUS TERURUT sempurna
Cara Kerja	Pemeriksaan satu per satu dari awal	Bandingkan nilai tengah, buang separuh
Langkah Kasus Terburuk	n Langkah	Kira-kira log2(n) + 1 Langkah
Kesesuaian Skenario	Data kecil / sekali cari pada data acak	Data besar, terurut, dan sering dicari

Perbandingan Lonjakan Langkah Kasus Terburuk:

- 16 Data : Linear = 16 Langkah | Binary Search = 5 Langkah
- 1.024 Data : Linear = 1.024 Langkah | Binary Search = 11 Langkah
- 1.000.000 Data: Linear = 1 Juta Langkah | Binary Search = 20 Langkah HANYA!

Kesimpulan: Untuk 1 juta data, Binary Search 50.000 kali lebih cepat dibanding Linear Search!

Flowchart dan Pseudocode Membantu Memetakan Logika Sebelum Coding

Simbol Standar Flowchart

- Terminator (Oval): Mulai atau Selesai.
- Proses (Persegi Panjang): Langkah pengolahan (misal menukar data).
- Keputusan (Belah Ketupat): Pertanyaan ya/tidak (misal A[j] > A[j+1]?).
- Input/Output (Jajar Genjang): Memasukkan atau menampilkan data.
- Alur (Panah): Arah langkah berikutnya.

Pseudocode Bubble Sort

```
ULANGI
    ada_tukar = salah
    UNTUK j dari 0 sampai n-2
        JIKA A[j] > A[j+1] MAKA
            tukar A[j] dan A[j+1]
            ada_tukar = benar
SAMPAI ada_tukar = salah

Aturan Menggambar Flowchart:
• Memiliki 1 titik Mulai & minimal 1 Selesai.
• Setiap Keputusan memiliki cabang 'Ya' dan
'Tidak'.
```

Array 2D Menyusun Data Seragam dalam Bentuk Baris dan Kolom

Konsep Array 2D (Matriks)

- Array 2D adalah array yang elemennya adalah array lagi (list di dalam list).
- Data tersusun dalam baris dan kolom seperti tabel matematika/excel.
- Format Akses: M[baris][kolom] (Ingat, indeks selalu mulai dari 0!).
- Urutannya selalu BARIS dahulu, baru KOLOM.
- Kelebihan: Sangat rapi untuk data tabel seragam dan mudah diolah per baris/kolom.
- Keterbatasan: Penambahan kolom menggeser indeks & arti kolom tersirat (harus diingat).

Contoh: Nilai 3 Siswa x 3 Mapel

```
M = [
  [80, 75, 90],   # Baris 0 (Siswa 1)
  [70, 85, 60],   # Baris 1 (Siswa 2)
  [95, 88, 92]    # Baris 2 (Siswa 3)
]

print(M[1][2])    # Output: 60 (Baris 1, Kolom 2)
M[1][2] = 65      # Mengubah nilai tersebut
print(M[0][1])    # Output: 75 (Baris 0, Kolom 1)

Hitung Rata-Rata Tiap Siswa:
for baris in M:
    print(sum(baris) / len(baris))
```

Dictionary Memanggil Data Menggunakan Label Unik (Key) yang Bermakna

Konsep Key-Value & Operasi

```
aset = {'nama': 'Proyektor', 'jumlah': 3}
```

- `Memanggil: aset['nama'] -> 'Proyektor'`
- `Mengubah : aset['jumlah'] = 4`
- `Menambah : aset['lokasi'] = 'Lab 1'`
- `Menghapus: del aset['lokasi']`
- `Memeriksa: 'nama' in aset`
- `Cadangan : aset.get('kondisi', '-')`

⚠️ **ATURAN KUNCI: Key HARUS UNIK! Jika menulis key yang sama 2 kali, nilai lama tertimpa tanpa peringatan. Memanggil key yang tidak ada memicu KeyError.**

Perbedaan: Akses Indeks vs Key

- Cara Memanggil: Nomor posisi M[1][2] VS Nama label aset['jumlah'].
- Arti Data: Tersirat pada posisi (harus diingat) VS Tertulis jelas pada label.
- Penyisipan Data: Indeks bergeser (risiko eror) VS Key tetap sama & aman.
- Kesesuaian: Data seragam berurutan VS Data berlabel dengan atribut bervariasi.

Struktur Bersarang dan JSON Solusi untuk Pengelolaan Data Kompleks

Dictionary Berkunci NIS/Kode

```
data = {
  'S001': {'nama': 'Ani', 'nilai':
{'Informatika': 85}},
  'S002': {'nama': 'Budi', 'nilai':
{'Informatika': 92}}
}

# Memanggil nilai Informatika S002:
print(data['S002']['nilai']['Informatika'])
# Output: 92

Keunggulan: Langsung menuju objek tanpa perlu
menelusuri satu per satu!
```

Sekilas tentang Format JSON

JSON (JavaScript Object Notation) adalah format teks standar untuk menyimpan dan menukar data terstruktur antar aplikasi daring.

Aturan Singkat JSON:
- Key dan teks WAJIB memakai tanda kutip dua (").
- Nilai logika ditulis true / false (huruf kecil).
- Di Python, dapat diubah dengan modul json.

Satu Objek = Satu Bungkus! Menambah atribut baru tidak menuntut variabel baru.

Pemilihan Struktur Data Harus Disesuaikan dengan Karakteristik Dataset

Karakteristik Data	Struktur Data Terpilih	Alasan Pemilihan
Semua baris punya kolom sama & berupa angka tabel	Array 2D (Matriks)	Rapi, hemat memori, mudah diolah per baris/kolom
Data berlabel & dipanggil menurut nama atribut	Dictionary	Label jelas, bebas dari risiko pergeseran posisi
Banyak objek berlabel & sering dicari via kode unik	Dictionary Berkunci Kode	Akses langsung 2 tingkat tanpa perlu penelusuran
Urutan tampilan sangat penting (papan peringkat)	List (atau List of Dict)	Mempertahankan urutan elemen secara konsisten

Contoh Peta Data: Katalog Aset Sekolah (A01 s.d. A04)

- `A01: Proyektor (3 unit, Lab 1) | A02: Laptop (30 unit, Lab 1) | A03: Router (5 unit, Server) | A04: Kabel LAN (40 unit, Gudang)`
- `Total Seluruh Aset = 78 Unit.`
- `Memanggil Jumlah Laptop: aset['A02']['jumlah'] -> Langsung bernilai 30!`

Desain Data yang Baik Harus Tahan Terhadap 6 Skenario Perubahan Mendadak

Skenario Uji Perubahan	Array 2D (Rancangan A)	Dictionary Berkunci Kode (Rancangan B)
Atribut Baru (misal: Tahun Beli)	Indeks kolom bergeser (risiko M[1][2] eror)	AMAN: Cukup tambah key 'tahun_beli'
Objek Baru Ditambahkan	Tambah baris di akhir tabel	AMAN: Tambah pasangan kode baru
Objek Dihapus	Baris bergeser, indeks objek lain berubah	AMAN: Hanya objek itu hilang, kode lain tetap
Data Bertambah 10x Lipat	Cari objek harus menelusuri baris 1 per 1	AMAN: Memanggil lewat kode tetap langsung
Dua Objek Kode Sama	Tidak ada error (ada 2 baris kembar)	BAHAYA: Objek lama tertimpa tanpa error
Salah Ketik Key / Indeks	Mengambil data salah tanpa error	BAGUS: KeyError muncul (kesalahan terlihat)

Rangkuman Konsep Kunci Pertemuan 11–13 & Penjelasannya

Rangkuman Poin Utama

✓ Indeks Selalu Mulai dari 0: Berlaku untuk Array, List, dan Array 2D. Elemen terakhir berada pada indeks n - 1.

✓ Stack vs Queue: Stack berbasis LIFO (Top, Undo, Back), sedangkan Queue berbasis FIFO (Front/Rear, Antrean Cetak).

✓ Pencarian & Pengurutan: Bubble Sort lambat (n^2); Linear Search untuk data acak; Binary Search sangat cepat (log n) tapi WAJIB TERURUT.

✓ Array 2D vs Dictionary: Array 2D untuk tabel angka seragam M[baris][kolom]; Dictionary untuk data berlabel key-value.

Klarifikasi Miskonsepsi Umum Siswa

• Miskonsepsi 1: 'Indeks array dimulai dari 1.' -> SALAH. Pada Python & mayoritas bahasa, indeks selalu dimulai dari 0.

• Miskonsepsi 2: 'pop() mengambil data paling awal.' -> SALAH. Pada stack, pop() mengambil data paling akhir diletakkan (Puncak).

• Miskonsepsi 3: 'Binary search bisa dipakai pada data acak.' -> SALAH. Pada data acak, hasilnya bisa salah (dapat menghilangkan data).

• Miskonsepsi 4: 'Key dictionary boleh sama.' -> SALAH. Key harus unik, jika sama nilai lama akan tertimpa!